

```
#include <iostream>

#include <string>

using namespace std;


string production[10];
char nonTerminal[10];
string firstSet[10];
int n;


void findFirst(char ch)
{
    for (int i = 0; i < n; i++)
    {
        if (nonTerminal[i] == ch)
        {
            string rhs = production[i];

            for (int j = 0; j < rhs.length(); j++)
            {
                char symbol = rhs[j];

                if (symbol >= 'a' && symbol <= 'z' || symbol == '#')
                {
                    if (firstSet[i].find(symbol) == string::npos)
                        firstSet[i] += symbol;

                    break;
                }
            }
        }
    }
}
```

```

    }

    else

    {

        findFirst(symbol);

    }

}

}

}

}
```

```
int main()
{
    cout << "Enter number of productions: ";

    cin >> n;

    cout << "Use # for epsilon\n";

    cout << "Enter productions (example: A=a|# )\n";

    for (int i = 0; i < n; i++)
    {
        cin >> production[i];

        nonTerminal[i] = production[i][0];

        production[i] = production[i].substr(2); // remove A=

    }

    for (int i = 0; i < n; i++)
```

```

{
    findFirst(nonTerminal[i]);
}

    cout << "\nFIRST Sets:\n";
for (int i = 0; i < n; i++)
{
    cout << "FIRST(" << nonTerminal[i] << ") = { ";
    for (int j = 0; j < firstSet[i].length(); j++)
    {
        cout << firstSet[i][j];
        if (j != firstSet[i].length() - 1)
            cout << ", ";
    }
    cout << " }\n";
}

return 0;
}

```

2.

```
#include <iostream>
```

```
#include <string>
```

```
using namespace std;
```

```
string prod[10];
```

```
char nt[10];
```

```
string followSet[10];
```

```
int n;
```

```
int indexOf(char c)
```

```
{
```

```
    for (int i = 0; i < n; i++)
```

```
        if (nt[i] == c)
```

```
            return i;
```

```
    return -1;
```

```
}
```

```
int main()
```

```
{
```

```
    cout << "Enter number of productions: ";
```

```
    cin >> n;
```

```
    cout << "Use # for epsilon\n";
```

```
    cout << "Enter productions (example: A=a|#)\n";
```

```
for (int i = 0; i < n; i++)  
{  
    cin >> prod[i];  
    nt[i] = prod[i][0];  
    prod[i] = prod[i].substr(2);  
}
```

```
followSet[0] = "$";
```

```
for (int i = 0; i < n; i++)  
{  
    for (int j = 0; j < n; j++)  
    {  
        string rhs = prod[j];  
        for (int k = 0; k < rhs.length(); k++)  
        {  
            if (rhs[k] == nt[i])  
            {  
                if (k + 1 < rhs.length())  
                {  
                    char next = rhs[k + 1];  
                    if (next >= 'a' && next <= 'z')  
                    {  
                        if (followSet[i].find(next) == string::npos)  
                            followSet[i] += next;  
                    }  
                }  
            }  
        }  
    }  
}
```



```

{
    cout << "FOLLOW(" << nt[i] << ") = { ";
    for (int j = 0; j < followSet[i].length(); j++)
    {
        cout << followSet[i][j];
        if (j != followSet[i].length() - 1)
            cout << ", ";
    }
    cout << " }\n";
}

return 0;
}

```

3.

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```

{
    cout << "LL(1) Parsing Table\n\n";

    cout << "M[E, id] = E -> T E'\n";
    cout << "M[E, +] = E' -> + T E'\n";
    cout << "M[E, $] = E' -> ε\n";
}

```

```
cout << "M[T, id] = T -> id\n";
```

```
return 0;
```

```
}
```